



Universidade Federal da Paraíba – UFPB
Campus IV – Rio Tinto – PB
Centro de Ciências Aplicadas à Educação – CCAE
Disciplina: Análise e Projetos de Sistemas
Professor: Dr. Dorgival Pereira da Silva Netto



Aluno: Lucas Henrique da Silva Menezes

DOCUMENTO DE ARQUITETURA DE SOFTWARE

SISTEMA DE GESTÃO DE FONTE DE ÁGUA

Disciplina: Programação Orientada a Objetos / Análise e Projeto de Sistemas

Professor: Dr. Dorgival Netto

Contexto do Sistema: Gestão de Fornecimento Local, Redes de Distribuição por Casas e Faturamento de Caminhões-Pipa.

Definição da Arquitetura do Sistema

Para o desenvolvimento do Sistema de Gestão de Fonte de Água, foi adotada uma arquitetura estruturada em camadas fundamentada no padrão de projeto arquitetural **MVC (Model-View-Controller)**, integrada à camada isolada de persistência **Repository (Data Access Object - DAO)**.

A escolha dessa estratégia arquitetural justifica-se pela necessidade de garantir uma separação clara de responsabilidades (Separation of Concerns), o que promove alta coesão em cada módulo e baixo acoplamento entre os componentes do sistema. Dessa forma, as regras de negócio permanecem completamente independentes das interfaces de usuário e dos mecanismos de armazenamento de dados.

O sistema é segmentado geograficamente e logicamente em quatro camadas principais:

Model (Modelo)

Camada responsável por representar as entidades de negócio do domínio e conter o estado dos dados do sistema. No escopo deste projeto, engloba as estruturas centrais:

- **Casa:** Entidade que mapeia as residências atendidas, contendo identificação única, dados do responsável e o estado operacional da sua ligação à rede.
- **Mensalidade:** Armazena o histórico financeiro, os meses de referência e o status de quitação de cada residência.
- **ViagemPipa:** Entidade que registra de forma imutável cada carregamento individual de caminhão-pipa realizado na fonte, atrelando a identificação do veículo e aplicando o valor fixo tarifado de R\$ 20,00 por viagem.
- **StatusRede:** Tipo enumerado (<<enumeration>>) responsável por restringir os estados válidos da infraestrutura hidráulica residencial (ATIVA, SUSPENSA, EM MANUTENÇÃO).

View (Visão)

Camada que compreende a interface de interação direta com o operador do sistema por meio de menus baseados em console/terminal Java. Sua única responsabilidade é capturar as entradas fornecidas pelo usuário e exibir de forma organizada as respostas, relatórios de faturamento e dados de consulta gerados pelas camadas internas. É subdividida em MenuPrincipalView (gerenciamento do fluxo de navegação primário) e GerenciamentoFonteView (interações específicas do domínio).

Controller (Controle)

Camada intermediária encarregada de coordenar o fluxo operacional do sistema. O Controller recebe as requisições geradas na camada de View, processa-as invocando as regras de validação de negócio apropriadas e aciona a camada de persistência para consolidação. Estão mapeados o CasaController (controle de faturamento e quitação residencial) e o ViagemPipaController (cálculo de receita global das fontes e liberação de carregamentos).

Repository / DAO (Acesso a Dados)

Camada dedicada exclusivamente ao isolamento dos mecanismos de armazenamento. Em conformidade com as diretrizes do projeto, esta camada abstrai as operações de escrita e leitura de dados utilizando persistência em arquivos locais padronizados (JSON/CSV) ou banco de dados embutido leve (SQLite). Centraliza métodos CRUD específicos nas classes CasaRepository e ViagemPipaRepository, assegurando que nenhuma regra de negócio ou tela acesse o meio de persistência de forma direta.

Benefícios e Justificativas Técnicas

A adoção do padrão MVC combinado ao padrão Repository provê vantagens cruciais para o ciclo de vida do software:

1. **Facilidade de Manutenção e Evolução:** Como cada camada executa funções delimitadas e exclusivas, correções e atualizações em regras de faturamento ou no formato de telas da View podem ser realizadas de forma cirúrgica, sem o risco de impactar ou corromper o comportamento de outras partes do software.
2. **Independência Tecnológica da Persistência:** A inserção das classes de Repository atua como uma barreira de isolamento. Caso o grupo opte inicialmente pelo armazenamento simplificado em arquivos locais JSON/CSV e futuramente decida migrar para um banco de dados relacional robusto (como MySQL ou PostgreSQL), a alteração ficará restrita estritamente ao corpo dos métodos do repositório. Os controladores e as telas da visão permanecerão intactos, pois dependem apenas de contratos abstratos.
3. **Consistência e Integridade das Regras de Negócio:** Centralizar fluxos lógicos nos Controllers (como a validação e o registro do valor imutável de R\$ 20,00 por viagem de pipa) impede a dispersão de código redundante e garante que os dados sejam permanentemente submetidos a validações rigorosas antes de sua efetiva gravação.

Próximos Passos do Ciclo de Desenvolvimento

Em alinhamento com o cronograma estabelecido na disciplina, as atividades subsequentes estão focadas na evolução deste design preliminar:

- **Aplicação de Padrões GRASP e GoF:** Refinamento da atribuição de responsabilidades das classes de controle e persistência.
- **Mapeamento de Diagramas de Sequência:** Detalhamento visual da troca dinâmica de mensagens entre os objetos das camadas View, Controller e Repository para validação dos fluxos.
- **Implementação Técnica do Protótipo:** Codificação completa em Java respeitando estritamente os pacotes arquiteturais aqui delineados.

